

Python — La visualisation de données

Document 70 — Images et texte

Série « Visualisation » · images, overlays et composition

Table des matières

- 1. Deux mondes : pixels et figures
- 2. Première image : afficher un fichier
 - 2.1 Avec Matplotlib (imshow)
 - 2.2 Avec Pillow (PIL.Image)
- 3. Le texte dans une figure
- 4. Les overlays : superposer sur une image
- 5. Dessiner avec Pillow
- 6. Composer image et graphique
- 7. Sauvegarde et formats
- 8. Matplotlib ou Pillow : lequel choisir
- Récapitulatif
- Questions de vérification

1. Deux mondes : pixels et figures

Jusqu'ici, on a tracé des données. Ce document traite l'inverse et le complément : afficher des images existantes (un logo, une capture, une carte), y superposer du texte et des repères, et combiner images et graphiques. Deux outils se partagent le terrain, et il faut savoir lequel pour quoi.

La distinction clé : Matplotlib raisonne en *figures* (axes, coordonnées de données) ; Pillow raisonne en *pixels* (une grille de points colorés). Afficher une image dans un graphique annoté relève de Matplotlib ; manipuler les pixels d'un fichier relève de Pillow.

Outil	Raisonne en
Matplotlib	Figures / axes
Pillow (PIL)	Pixels

Pourquoi ça compte. La question n'est pas « lequel est meilleur » mais « pixels ou figure ». On compose une planche annotée avec Matplotlib ; on retouche un fichier image avec Pillow. Souvent on enchaîne : Pillow prépare l'image, Matplotlib la met en scène.

2. Première image : afficher un fichier

2.1 Avec Matplotlib (imshow)

Pour montrer une image dans une figure — afin de l'annoter ou de la placer à côté d'un graphique — on la lit puis on l'affiche avec `imshow`. C'est un tracé comme un autre, sur un Axes.

```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

```
img = mpimg.imread("logo.png")      # tableau de pixels
fig, ax = plt.subplots()
ax.imshow(img)
ax.axis("off")                        # masquer les axes
plt.show()
```

`ax.axis("off")` masque les graduations, qui n'ont pas de sens sur une image. `imshow` accepte aussi des données numériques (une matrice) et les colore : c'est le principe de la heatmap.

2.2 Avec Pillow (PIL.Image)

Pour travailler l'image elle-même — taille, recadrage, format — on l'ouvre avec Pillow, qui renvoie un objet Image manipulable.

```
from PIL import Image

img = Image.open("photo.jpg")
print(img.size, img.mode)          # (largeur, hauteur), mode couleur
petite = img.resize((200, 150))    # redimensionner
recadree = img.crop((0, 0, 100, 100))# recadrer
```

Pourquoi ça compte. `imread/imshow` de Matplotlib pour afficher et annoter dans une figure ; `Image.open` de Pillow pour ouvrir et transformer le fichier. Les deux lisent les mêmes PNG/JPG, mais n'en font pas la même chose.

3. Le texte dans une figure

Au-delà du titre et des libellés (document 10), on ajoute du texte libre dans une figure : une note, une signature, un commentaire. Deux systèmes de coordonnées coexistent, et les distinguer est essentiel.

Méthode	Coordonnées
<code>ax.text(x, y, s)</code>	Données
<code>ax.annotate(...)</code>	Données
<code>fig.text(x, y, s)</code>	Figure (0–1)
<code>transform=ax.transAxes</code>	Axes (0–1)

```
fig, ax = plt.subplots()
ax.plot(jours, prix)

# note calée sur la donnée
ax.text(3, 105, "Cassure", fontsize=11, color="darkred")

# note calée sur le cadre (coin haut-gauche), quelle que soit la donnée
ax.text(0.02, 0.95, "Source : interne", transform=ax.transAxes,
        fontsize=9, color="grey", va="top")

# signature globale en bas de figure
fig.text(0.99, 0.01, "ES – 2024", ha="right", color="grey")
```

Données ou cadre. `transform=ax.transAxes` est le réflexe clé : il place le texte en proportion de l'axe (0 à 1), indépendamment des valeurs. Une mention « Source » doit rester dans le coin même si les prix changent — donc en coordonnées d'axes, pas de données.

4. Les overlays : superposer sur une image

Un overlay, c'est dessiner par-dessus une image affichée : encadrer une zone, pointer un détail, ajouter une légende. Avec Matplotlib, on superpose simplement d'autres tracés sur le même Axes après `imshow` — l'ordre d'appel détermine la superposition.

```
import matplotlib.patches as mpatches

fig, ax = plt.subplots()
ax.imshow(img)

# encadrer une zone (rectangle)
rect = mpatches.Rectangle((50, 30), 80, 60,
                           edgecolor="red", facecolor="none", linewidth=2)
ax.add_patch(rect)

# pointer un détail
ax.annotate("Anomalie", xy=(90, 60), xytext=(150, 20),
            color="red", arrowprops=dict(arrowstyle="->", color="red"))
ax.axis("off")
```

Élément	Rôle
<code>imshow(img)</code>	Affiche l'image, en arrière-plan.
<code>add_patch(Rectangle)</code>	Encadre une zone (sélection, détection).
<code>annotate(...)</code>	Flèche et texte vers un point de l'image.
<code>plot(x, y)</code> par-dessus	Trace des repères (points détectés, contour).

Pourquoi ça compte. Sur un Axes, tout ce qu'on dessine après `imshow` se pose au-dessus de l'image. Cadres, flèches et points partagent le repère en pixels de l'image : un overlay Matplotlib n'est qu'une superposition de tracés ordinaires.

5. Dessiner avec Pillow

Quand on veut écrire dans le fichier image lui-même — un filigrane, un cachet, une annotation gravée dans les pixels — c'est Pillow et son module `ImageDraw`. Le résultat fait partie de l'image, pas d'une figure par-dessus.

```
from PIL import Image, ImageDraw, ImageFont

img = Image.open("graphique.png").convert("RGB")
draw = ImageDraw.Draw(img)

draw.rectangle([50, 30, 130, 90], outline="red", width=3)
draw.text((50, 95), "CONFIDENTIEL", fill="red")
img.save("graphique_tampon.png")    # le texte est dans les pixels
```

Objet / méthode	Rôle
<code>ImageDraw.Draw(img)</code>	Ouvre un « pinceau » sur l'image.
<code>draw.line / rectangle</code>	Trace des formes dans les pixels.
<code>draw.text(...)</code>	Écrit du texte (filigrane, cachet).
<code>ImageFont.truetype(...)</code>	Charge une police et une taille précises.

Gravé, pas superposé. Différence fondamentale avec Matplotlib : ce que dessine ImageDraw modifie réellement les pixels du fichier. Idéal pour un filigrane indissociable de l'image ; à éviter si l'on veut pouvoir retirer l'annotation.

6. Composer image et graphique

Le cas concret le plus fréquent : une planche mêlant un logo, un graphique de données et du texte. On la compose avec une grille de subplots (document 10), où une case reçoit l'image et une autre le tracé.

```
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(10, 5))
gs = gridspec.GridSpec(1, 2, width_ratios=[1, 3])

ax_logo = fig.add_subplot(gs[0])
ax_logo.imshow(logo)
ax_logo.axis("off")

ax_data = fig.add_subplot(gs[1])
ax_data.plot(jours, prix)
ax_data.set_title("Performance ES")

fig.suptitle("Rapport quotidien", fontsize=14)
fig.savefig("rapport.pdf")
```

Pourquoi ça compte. Une planche professionnelle n'est qu'un assemblage de subplots : une case pour l'image (axis off), une pour le graphique, un supitle pour coiffer le tout. On réutilise directement GridSpec du document 10 — afficher une image est un tracé comme un autre.

7. Sauvegarde et formats

Selon l'outil, on sauvegarde différemment, mais les formats et leurs usages restent ceux du document 10.

Côté Matplotlib. `fig.savefig`, comme partout dans la série : PNG (écran), PDF/SVG (vectoriels, impression). L'image affichée par `imshow` est aplatie dans la figure.

Côté Pillow. `img.save`, qui déduit le format de l'extension et offre des options fines (qualité JPEG, optimisation PNG).

```
img.save("sortie.png")           # PNG sans perte
img.save("sortie.jpg", quality=90) # JPEG, qualite réglable
img.save("planche.pdf")          # Pillow écrit aussi le PDF
```

Format	Quand l'utiliser
PNG	Captures, graphiques, transparence : sans perte.
JPG	Photographies ; éviter pour les graphiques (artefacts).
PDF / SVG	Vectériel, pour l'impression et les documents.
GIF	Petites animations (lien avec Matplotlib animation, doc 50).

Le réflexe couleur. PNG pour tout ce qui contient du texte net, des aplats ou de la transparence (donc les graphiques et captures) ; JPG réservé aux photos. C'est le même conseil qu'au document 10, valable des deux côtés.

8. Matplotlib ou Pillow : lequel choisir

Récapituler le partage des rôles évite bien des détours. La question à se poser : veut-on une mise en scène (figure annotée) ou une retouche de pixels (fichier modifié) ?

Besoin	Outil
Afficher une image dans un graphique	Matplotlib (imshow).
Annoter de façon réversible	Matplotlib (overlays sur l'Axes).
Composer une planche image + données	Matplotlib (subplots / GridSpec).
Redimensionner, recadrer, convertir	Pillow (resize, crop, save).
Graver un filigrane dans les pixels	Pillow (ImageDraw).
Traiter des images en masse (batch)	Pillow (boucle de fichiers).

Pourquoi ça compte. Règle simple : Matplotlib pour mettre une image en scène dans une figure (réversible, annotable) ; Pillow pour transformer le fichier lui-même (gravé, automatisable). Le tandem couvre tout l'éventail « images + texte » d'un projet de visualisation.

Récapitulatif

- Deux mondes.** Matplotlib raisonne en figures (axes, données) ; Pillow en pixels (le fichier image).
- Afficher.** imshow (dans une figure, pour annoter) ou Image.open (pour transformer le fichier).
- Le texte.** ax.text/annotate en coordonnées de données ; transform=ax.transAxes pour caler sur le cadre.
- Les overlays.** Après imshow, tout tracé (Rectangle, flèche, points) se superpose à l'image sur le même Axes.
- Pillow ImageDraw.** Écrit dans les pixels (filigrane, cachet) : gravé, non superposé.
- Composer.** Une planche = des subplots (GridSpec) : une case image (axis off), une case graphique, un supitle.
- Export.** fig.savefig (Matplotlib) ou img.save (Pillow) ; PNG pour les graphiques, JPG pour les photos.
- Le choix.** Mise en scène réversible -> Matplotlib ; retouche de pixels automatisable -> Pillow.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Quelle est la distinction fondamentale entre Matplotlib et Pillow pour les images ?

Réponse. Matplotlib raisonne en figures (axes, coordonnées de données) : il affiche une image pour l'annoter ou la composer. Pillow raisonne en pixels : il ouvre, redimensionne, dessine et écrit le fichier image lui-même.

Q2. Comment affiche-t-on une image dans une figure Matplotlib, et pourquoi `axis("off")` ?

Réponse. Avec `ax.imshow(img)` après l'avoir lue (`mpimg.imread`). On appelle `ax.axis("off")` pour masquer les graduations, qui n'ont pas de sens sur une image (contrairement à un graphique de données).

Q3. À quoi sert `transform=ax.transAxes` dans un `ax.text` ?

Réponse. À placer le texte en coordonnées d'axes (0 à 1), calées sur le cadre du graphique et non sur les valeurs. Une mention « Source » reste ainsi dans le coin même si les données changent, alors que des coordonnées de données la déplaceraient.

Q4. Comment réalise-t-on un overlay (encadrer une zone) sur une image avec Matplotlib ?

Réponse. On affiche l'image avec `imshow`, puis on dessine par-dessus sur le même Axes : `ax.add_patch(Rectangle(...))` pour un cadre, `annotate` pour une flèche. Tout ce qui est tracé après `imshow` se superpose, dans le repère en pixels de l'image.

Q5. Quelle différence entre annoter avec Matplotlib et dessiner avec Pillow `ImageDraw` ?

Réponse. Matplotlib superpose l'annotation dans une figure : elle reste séparée de l'image et réversible. `ImageDraw` grave le dessin dans les pixels du fichier : c'est permanent, indissociable de l'image — idéal pour un filigrane.

Q6. Comment compose-t-on une planche mêlant logo, graphique et titre ?

Réponse. Avec une grille de subplots (`GridSpec`, document 10) : une case reçoit le logo via `imshow` + `axis("off")`, une autre le graphique de données, et `fig.suptitle` coiffe l'ensemble. Afficher une image est un tracé comme un autre.

Document suivant (999) : *Cheat sheets — la référence transversale : colormaps, marqueurs, styles de ligne, rcParams et patterns fréquents de toute la série.*